MACHINE LEARNING • PROJECT DOSSIER

ChurnGuard

Customer Churn Prediction Service

An end-to-end ML service: data → features → training → evaluation → a served REST API, built with production practices. This dossier: what it is, how it works, and interview Q&A.

Pratham Shukla
prathamshukla2804@gmail.com
github.com/prathamshukla28/churnguard

Python · scikit-learn · FastAPI · Docker · CI

1 What ChurnGuard Is

ChurnGuard is a production-style machine-learning service that predicts which telecom customers are about to cancel their subscription, so a retention team can intervene *before* they leave. It is not a notebook — it is a packaged, tested, deployable service that covers the full ML lifecycle end to end.

0.81	0.129	79%	393
ROC-AUC	Brier score	Recall @ 0.20	Cost saved / 1k

The one-line pitch

"I built an end-to-end churn-prediction service: a leak-free scikit-learn pipeline that is cross-validated, hyperparameter-tuned, probability-calibrated, and served behind a FastAPI REST API with per-prediction SHAP explanations, Docker packaging, a test suite, and CI."

Why churn prediction?

For a subscription business, keeping an existing customer is far cheaper than acquiring a new one. A model that ranks customers by churn risk lets a small retention budget be spent where it matters most. The project frames the problem the way a business actually experiences it — as a **cost trade-off** between chasing customers who were going to stay anyway (false positives) and missing customers who actually leave (false negatives).

The problem, precisely

- **Task:** binary classification — will this customer churn? Output a calibrated probability, not just a yes/no.
- **Data:** ~7,043 Telco customers with ~21% base churn rate (class imbalance). Features include tenure, contract type, monthly/total charges, internet service, payment method and support add-ons.
- **Success metric:** ROC-AUC and PR-AUC for ranking quality, Brier score for probability calibration, and — crucially — **expected business cost** at the chosen decision threshold.

2 Architecture & Data Flow

Every stage is a small, single-responsibility module driven by one config file (`config/config.yaml`) — there are no magic numbers buried in the code.

```
config/config.yaml (single source of truth)
|
data.py -> features.py -> train.py ------------> evaluate.py
(real or (engineering + (CV select -> (threshold choice,
synth) ColumnTransformer) tune -> calibrate) metrics, figures)
|
v
models/churnguard.joblib (ModelBundle: model + threshold + SHAP bg)
|
+--> explain.py (SHAP) predict.py -> api.py
(batch CLI) (/health /metrics /stats /predict)
|
v
Docker image -> Render / HF Spaces / anywhere
```

Key design choices

- **No data leakage.** All scaling and encoding live inside a scikit-learn `Pipeline`, fitted on the training fold only and cross-validated as one unit.
- **Train/serve parity.** The same `engineer_features()` runs at training time and at request time, so the model never sees differently shaped inputs.
- **Self-describing artifact.** The saved `ModelBundle` carries the model, the business threshold, the feature schema and a SHAP background sample — the service is a single file.
- **Reproducible.** One seed flows through generation, splitting and modelling; `requirements.lock` pins the exact dependency tree.

3 Technical Deep-Dive

3.1 Features & preprocessing

A `ColumnTransformer` applies `SimpleImputer + StandardScaler` to numeric columns and `SimpleImputer + OneHotEncoder(handle_unknown='ignore')` to categoricals. Wrapping it in a `Pipeline` guarantees the scaler's mean/variance are learned only from training data — the single most common source of leakage in tabular ML.

3.2 Model selection, tuning, calibration

- **Compare:** logistic regression, random forest and gradient boosting are scored by 5-fold stratified cross-validated ROC-AUC. The winner here is **logistic regression** (CV AUC 0.809).
- **Tune:** `RandomizedSearchCV` on the winner (best `C=0.071`).
- **Calibrate:** `CalibratedClassifierCV` (isotonic) so a predicted “0.7” really means ~70% — Brier score 0.129.

3.3 The business threshold (the standout idea)

The default 0.5 cutoff is arbitrary. ChurnGuard sweeps every threshold from 0.01 to 0.99 and picks the one that **minimises expected cost**, given that a missed churner (false negative) costs 5× a wasted retention offer (false positive). The chosen threshold of **0.20** cuts expected cost from **1047** to **654** per 1,000 customers (~37% lower) versus the naive 0.5 cutoff. This is the detail that turns a model into a decision system.

3.4 Explainability (SHAP)

Every prediction returns its top risk drivers. For the linear model, SHAP values have a closed form (`coef · (x − E[x])`); for tree models the code uses `shap.TreeExplainer`. Transformed one-hot columns are mapped back to business-readable names, so the API returns “`Contract = Month-to-month increases risk`”, not a cryptic encoded column.

3.5 Serving, MLOps & quality

- **FastAPI** service: `/` (interactive demo), `/health`, `/metrics`, `/stats`, `/predict` (with an `explain` flag).
- **Batch CLI:** `python -m churnguard.predict in.csv --out scored.csv`.
- **Docker** packaging with a container HEALTHCHECK; binds `$PORT` for Render / HF Spaces.
- **CI (GitHub Actions):** ruff + mypy + pytest with coverage on Python 3.10 and 3.12, plus an end-to-end train→evaluate smoke test.
- **Tracking:** best-effort MLflow logging of every training and evaluation run.

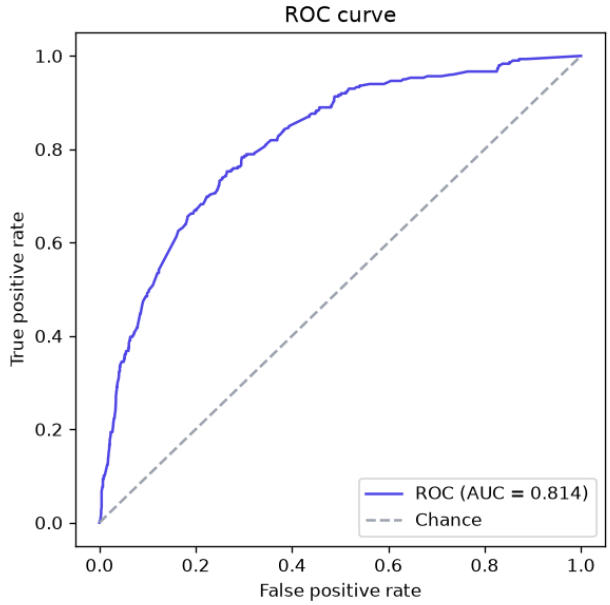
- **Quality gate:** 25 tests, ~89% coverage, clean ruff + mypy.

4 Results

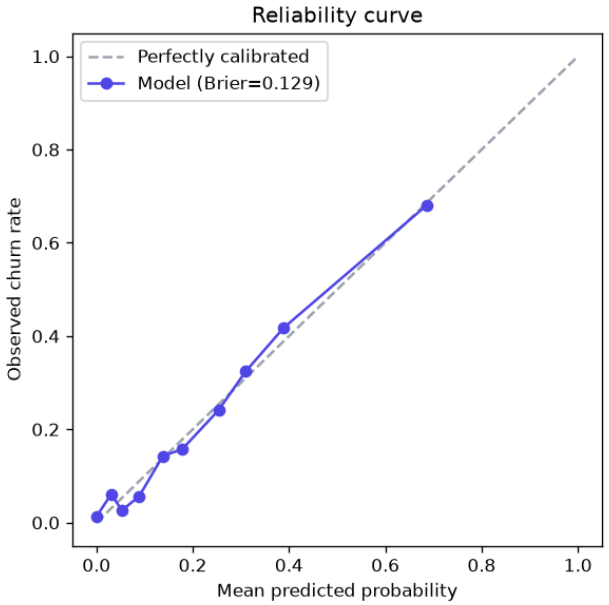
Trained on the Telco-style dataset (1409 held-out test customers, 21% churn), the tuned and calibrated model reaches:

Metric	Value	What it means
ROC-AUC	**0.8144**	Ranks a random churner above a random non-churner ~81% of the time
PR-AUC	**0.5376**	Strong vs. the 21% base churn rate
Brier score	**0.1291**	Calibration quality of the probabilities (lower is better)
Recall	**0.79**	Catches 79% of true churners at the chosen threshold
Precision	**0.41**	Of flagged customers, 41% truly churn
F1	**0.54**	Harmonic mean of precision and recall
Threshold	**0.20**	Chosen to minimise expected business cost (not 0.5)
Expected cost	**654 vs 1047**	~37% lower than the naive 0.5 cutoff (cost_fn=5, cost_fp=1)

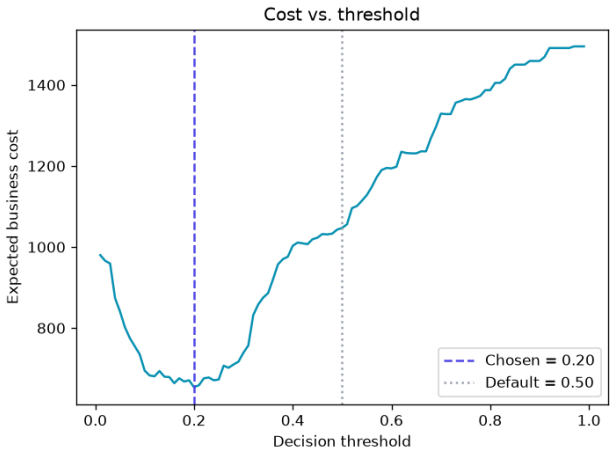
Evaluation figures

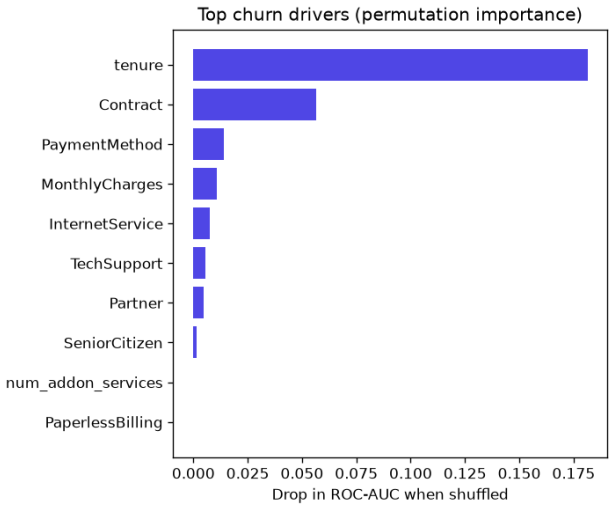


ROC curve



Reliability / calibration





Cost vs. threshold

Permutation importance

Top churn drivers (permutation importance): **tenure**, **Contract**, **PaymentMethod**, **MonthlyCharges**, **InternetService** — matching the known real-world Telco churn story (new, month-to-month, electronic-check customers churn most).

5 Tech Stack

Layer	Tools
Language	Python 3.10+
ML / data	scikit-learn, pandas, NumPy, SHAP
Serving	FastAPI, Uvicorn, Pydantic
Packaging	Docker, pip-tools (requirements.lock)
Quality	pytest + coverage, ruff, mypy, pre-commit
MLOps	File-based model registry, champion/challenger, CI regression gate
Monitoring	PSI drift (data/prediction/performance), MLflow tracking
Observability	JSON logs, Prometheus metrics, rate limiting, API-key auth
Ops	GitHub Actions CI + scheduled retrain, Makefile
Deploy	Render blueprint / Hugging Face Spaces

6 Production Operations & MLOps

Training a good model is the easy 20%. ChurnGuard is built to be *operated* — the post-deployment lifecycle most portfolios never show. Full rationale lives in the Architecture Decision Records (`docs/adr/`); the on-call playbook is in `docs/RUNBOOK.md`.

6.1 Drift monitoring

A model decays as the world shifts; one that is never watched is a liability. A live batch is compared to a **reference profile** captured at evaluation time, and drift is graded with the **Population Stability Index (PSI)** — the churn/credit-scoring standard (`<0.1` stable, `0.1–0.25` moderate, `>0.25` significant). It covers three kinds of drift:

- **Data drift** — PSI per input feature (the population changed).
- **Prediction drift** — PSI on the score distribution (outputs shifted).
- **Performance drift** — live ROC-AUC vs. the baseline, once labels arrive.

Exposed via `POST /monitor` and `python -m churnguard.monitor batch.csv`. Implemented in a few lines of NumPy on purpose — no framework, so it runs the moment the repo is cloned.

6.2 Model registry & champion/challenger

Every trained model is an **immutable, versioned artifact** with a lifecycle stage (`staging` → `production` → `archived`) in a manifest. Promoting a version to production atomically archives the incumbent, giving full provenance and **one-command rollback**. A scheduled retrain trains a **challenger** and promotes it to **champion** only if it beats production on the configured metric — otherwise production is left untouched.

6.3 CI quality gate

Model quality is a first-class CI check, not a surprise in production. On every push the pipeline trains and evaluates end to end, then a **regression gate** fails the build if ROC-AUC/PR-AUC fall — or Brier rises — beyond tolerance versus a committed baseline. A bad retrain is blocked exactly like a failing unit test.

6.4 Observability & hardening

- **Structured JSON logs** with a request id (returned as `X-Request-ID`) — traceable in any aggregator.
- **Prometheus** exposition at `/metrics/prometheus`; latency **p50/p95/p99** and throughput at `/stats`.
- **Per-IP rate limiting** and optional **API-key auth** (`X-API-Key`) on the scoring endpoints.
- A serving **benchmark** (`reports/loadtest.md`): p50 ~19 ms per single-row request, in-process.

All standard-library — zero new runtime dependencies — and each component is honest about its scope (per-process metrics for per-instance scraping; a gateway enforces global limits across replicas).

7 Interview Questions & Answers

Grouped from “walk me through your project” to deeper ML, system-design and MLOps probes. Answers are written the way you would actually say them out loud.

A. Project walkthrough

Q Walk me through this project end to end.

A ChurnGuard predicts telecom customer churn. Raw data is feature-engineered, then fed into a scikit-learn Pipeline so preprocessing can't leak. I compare three models by cross-validated ROC-AUC, tune the winner with randomized search, calibrate its probabilities with isotonic regression, and choose a decision threshold that minimises business cost. The model is saved as a self-describing bundle and served behind a FastAPI API that also returns a SHAP explanation per prediction. It's Dockerised, tested, and runs through CI.

Q What was the hardest part?

A Getting the evaluation honest. It's easy to report a high accuracy on a 21%-churn dataset — predicting ‘no churn’ for everyone scores 79%. So I moved to ROC-AUC, PR-AUC, calibration (Brier), and a cost-based threshold, which reflects what the business actually pays for mistakes.

B. Core ML concepts

Q Why is accuracy a bad metric here, and what did you use instead?

A With 21% churn, a model that always predicts ‘stay’ is 79% accurate but useless. I use ROC-AUC and PR-AUC for ranking quality, Brier score for calibration, and expected cost at the operating threshold for the business decision.

Q What is data leakage and how did you prevent it?

A Leakage is when information unavailable at prediction time sneaks into training — e.g. scaling with statistics computed over the whole dataset including the test rows. I prevent it by putting every transform inside a Pipeline that's fitted only on the training fold, and cross-validating the whole Pipeline as one unit.

Q What does probability calibration mean and why does it matter?

A A calibrated model's 0.7 means ~70% of such customers actually churn. It matters because the business acts on the probability (expected-value decisions, ranking by risk), not just the label. I use CalibratedClassifierCV with isotonic regression and verify it with the Brier score and a reliability curve.

Q How did you choose the decision threshold?

A Not 0.5. I assign costs — a missed churner costs 5x a wasted retention offer — then sweep thresholds and pick the one minimising total expected cost. Here that's 0.20, cutting cost by ~37% versus 0.5.

Q Precision vs. recall — which matters more here?

A Recall, within reason. Missing a churner (false negative) is the expensive mistake, so the cost ratio pushes the threshold down to catch ~79% of churners, accepting lower precision. The exact balance falls out of the cost function rather than a gut call.

Q How do you handle class imbalance?

A Stratified splits and stratified CV, threshold-based rather than 0.5 decisions, PR-AUC alongside ROC-AUC, and class weighting available in the estimators. I deliberately avoided blind oversampling, which can hurt calibration.

Q Why logistic regression over the tree ensembles?

A It genuinely won on cross-validated AUC on this data, and it's faster, more interpretable, and calibrates cleanly — its SHAP values even have a closed form. On this dataset the linear signal dominates; I let CV decide rather than assuming a fancier model is better.

C. Explainability & trust

Q How does SHAP work and how did you use it?

A SHAP attributes a prediction to each feature using game-theory Shapley values — each feature's average marginal contribution. For my linear model that reduces to coef x (x - mean), which I compute directly; for trees I'd use TreeExplainer. I map the transformed columns back to business names so the API returns readable reasons.

Q A stakeholder doesn't trust the model. What do you show them?

A The reliability curve (probabilities are honest), the global drivers (they match domain intuition — tenure, contract type, payment method), and a per-customer SHAP breakdown for a specific flagged account. Trust comes from the model agreeing with what they already know, then adding precision.

D. Engineering & system design

Q How is the model served and deployed?

A A FastAPI app loads the bundle once and exposes /predict with optional explanations, plus /health, /metrics and /stats. It's Dockerised with a healthcheck and binds $PORT so it drops straight onto Render or HF Spaces. A batch CLI reuses the exact same inference code for scoring CSVs.

Q How do you ensure the model behaves the same in training and production?

A Train/serve parity: one engineer_features() function used in both paths, a fixed feature schema stored in the bundle, and reindexing of incoming records to that schema with handle_unknown='ignore' on the encoder so unseen categories don't crash.

Q What does your CI pipeline do?

A On every push and PR it lints with ruff, type-checks with mypy, runs the test suite with coverage on Python 3.10 and 3.12, and runs a smoke test that trains and evaluates the full pipeline end to end and asserts the artifacts exist.

Q How would you monitor this model in production?

A Track input-feature drift and prediction-distribution drift, watch the realised cost once outcomes are known, and periodically re-fit the threshold as the true cost ratio changes. The /stats endpoint logging risk-band mix is the first step toward that.

Q How would you scale or extend it?

A Add a model registry with automated promotion on top of the MLflow runs, a feature store for parity at scale, batch scoring on a schedule, and a small dashboard over the scores. For throughput, the stateless API scales horizontally behind a load balancer.

E. Production ML & MLOps

Q How do you know if your model is still good after deployment?

A I monitor for drift. A live batch is compared to a reference profile from training using PSI: data drift on each input, prediction drift on the score distribution, and performance drift on ROC-AUC once labels arrive. PSI over 0.25 on a key feature is my signal to investigate an upstream data bug first, then retrain if it's genuine.

Q What is PSI and why use it over a KS test?

A Population Stability Index measures how much a distribution moved by bucketing both and summing (actual% - expected%) x ln(actual%/expected%). It's the churn/credit standard: thresholded, explainable to non-stats stakeholders, and works for numeric and categorical features the same way. KS is a fine complement for numeric features; I kept PSI because it's interpretable and needs only a tiny committed reference.

Q Walk me through your model registry and promotion.

A Every model is an immutable version with a stage - staging, production, archived - in a manifest. Retraining produces a challenger; it's promoted to production only if it beats the current champion on the chosen metric, and promotion archives the incumbent so rollback is one command. It's file-based rather than a hosted MLflow registry because one service doesn't need a server to run, and it's trivially testable.

Q How do you stop a bad model from reaching production?

A A CI regression gate. Every push trains and evaluates, then fails the build if ROC-AUC or PR-AUC drop - or Brier rises - beyond a tolerance versus a committed baseline. Model quality is checked like a unit test, so a bad retrain never ships.

Q How would you make retraining automatic?

A A scheduled job - cron, Airflow, or a GitHub Actions schedule, which is what I wired up - trains a challenger, runs the regression gate, and promotes only on improvement. The decision is codified in evaluate_promotion, so the same logic runs by hand, on a schedule, or in CI with no divergence.

F. Reflection

Q What would you do differently or add next?

A Train on the full real Kaggle dataset in CI, add drift monitoring, and A/B test the retention actions the model triggers so I can measure realised ROI, not just offline cost. I'd also add online threshold re-tuning as the cost ratio shifts.

Q What did this project teach you?

A That the modelling is the easy 20%. The value is in honest evaluation, calibration, a cost-aware decision, explainability, and the engineering around it — tests, CI, reproducibility and clean serving — that makes it something a team can actually run.

Pratham Shukla · prathamshukla2804@gmail.com · github.com/prathamshukla28/churnguard